When Does Predicting Failure Help?

A Boundary Analysis of Predictive versus Reactive Runtime Assurance for Multi-Robot Coordination

Mohamed Adil Farukh

mdadilfarooq@gmail.com

Overview

A prediction can tell a robot to act before a safety margin runs out. But a fixed threshold can also make it act early. I wanted to understand when prediction adds something beyond choosing a more conservative threshold. I study this question with a simplified timing model and two kinematic simulation setups, giving both rules the same tuning budget and testing on separate noise seeds.

In the timing model, the predictive rule is a current-margin threshold whose buffer scales with the estimated rate of decline. Extra lead changes the recovery outcome only within a band set by recovery duration and deadline slack; old observations consume that lead one for one. The minimal predictive horizon wastes less lead than the minimal safe fixed buffer when recovery needs a similar amount of time across cases but the corresponding margins differ, provided the rates can be estimated.

The clearest gain appears in the three-robot shared-zone test. Both tuned rules succeed in all 1,620 conflict episodes, and prediction reduces mean completion time by 2.78 s (paired 95% bootstrap interval $[2.66, 2.90]$ s). On clear paths, it reduces unnecessary serializations from 1,080 to 21 out of 1,620 episodes. The two-robot test exposes a different behavior: the tuned fixed rule records no collisions in 6,750 conflict episodes and prediction records two. Together, the tests show where adapting the margin helps in these simulations and where estimating its rate becomes the weak point.

Keywords: Runtime assurance, multi-robot systems, predictive monitoring, safety margins, time to collision, reproducible evaluation.

1 The question

A prediction can tell a robot to act before something goes wrong. But a simple threshold can also make it act early: increase the buffer and intervene while there is still room to recover. I wanted to understand when prediction adds something beyond choosing a more conservative threshold.

Runtime assurance provides the setting for this question. A runtime layer monitors a safety, performance, or mission condition and intervenes during execution. Examples include modular runtime assurance for robotics [1], mission-level assurance for autonomous driving [2], monitoring across robot swarms [3], and predictive verification, enforcement, and monitoring [4–7]. Earlier intervention may give recovery enough time to preserve the mission instead of requiring an emergency stop.

Prediction also has costs. The monitor has to estimate how fast a margin is shrinking from noisy, possibly delayed observations and finish within the robot's decision cycle. Comparing it only

with a reactive rule that waits until the margin is exhausted gives prediction an easy baseline. I instead compare it with a fixed-margin rule that has the same tuning budget. The question is: *when does predicting the loss of feasibility outperform reacting to the current margin?*

What I tried and what I found

I start with a simplified timing model (Section 4). The predictive trigger studied here turns out to be a current-margin threshold whose buffer scales with the estimated rate of decline. Earlier action changes the recovery outcome only within a band set by recovery duration and deadline slack. Old observations consume that lead one for one. The smallest predictive horizon wastes less lead than the smallest safe fixed buffer when recovery needs a similar amount of time across cases but, because rates differ, a different amount of margin.

I then test the mechanism in two simulation setups (Sections 5 and 6). Exploratory sweeps show where behavior changes. Separate tuning and testing runs use different noise seeds and give both rules equal tuning budgets. I check failures before comparing completion times.

The three-robot shared-zone test gives the clearest result: both tuned rules succeed in all 1,620 conflict episodes, while prediction reduces mean completion time by 2.779 s and avoids most unnecessary serializations on clear paths. The first intervention commits the robots to one crossing order, so choosing when to commit directly affects how long they wait.

The two-robot test helps explain the other side of the tradeoff. The tuned fixed rule records 0/6750 conflict collisions and the predictive rule records 2/6750, both in the slowest-recovery, late-reveal corner of the grid. I examine that behavior alongside the timing model to understand how rate estimation changes the lead actually available for recovery.

2 How this connects to prior work

Predictive runtime verification and enforcement. Predictive runtime verification uses a priori knowledge of the system to reach a conclusive verdict before the observed execution alone permits one [4], and predictive runtime enforcement reduces to non-predictive enforcement when no such knowledge is available [5]. For stochastic linear systems, Yoon *et al.* determine whether a control sequence exists over a finite horizon that maintains a safety property with sufficiently high probability [6]. RADICS combines a black-box monitor that switches to a safe algorithm on a failing trajectory with a white-box monitor that predicts such trajectories earlier [7]. Predictive safety filters solve an online finite-horizon problem to certify or modify a proposed input under model uncertainty [8]. I ask a narrower comparative question: once recovery timing and a tuned current-margin threshold are included, when does an earlier trigger change the measured outcome?

Runtime-assurance architectures. Simplex uses a simple, reliable core to bound the behavior of more complex functionality [9]. Its black-box extension uses runtime checks to transfer control between advanced and baseline controllers and includes a multi-robot case study [10], and SOTER composes robotics runtime-assurance modules that switch between advanced and safe controllers [1]. Mission-level monitors can reject commands that would make a mission infeasible before they execute [2]; compositional monitoring can detect policy violations that emerge only across a robot swarm [3]; and collective run-time adaptation addresses safety and mission completion in multi-robot systems [11]. These works design what happens after a trigger. I study the trigger rule itself: a projected time to threshold versus a fixed current-margin threshold.

Adaptive margins and time-to-threshold triggers. Time to collision, the ratio of range to closing speed, has long been used to time braking and to grade traffic conflicts [12,13]. Proposition 1 is the corresponding observation for a generic monitored margin: a threshold on projected time is a threshold on the margin scaled by the estimated rate. Adaptive margins can also scale a safety buffer with relative speed, braking capability, and prediction uncertainty [14], and control barrier functions provide a broader framework for encoding and enforcing safety constraints [15]. Because the horizon trigger studied here is one particular rate-scaled margin, it should be compared with tuned fixed margins and not only with an unpadded threshold.

Prognostics and recoverability. Prognostics estimates whether and when faults will occur in order to inform condition-based maintenance [16]. Runtime-assurance verification formalizes the adequacy of decision timing and time-bounded recoverability [17]. Recent work on adaptive governance of AI agents predicts when a viability index will cross its boundary from a first-order fit of its recent trend [18]. These results motivate a point-of-no-return analysis, but they do not by themselves establish when an earlier trigger beats a tuned fixed margin for a robot model.

3 The timing model and trigger rules

Let a scalar margin $M(t) \in \mathbb{R}$ summarize the remaining feasibility of a mission, with $M(t) \geq 0$ defining the monitored feasible set and $M(t) = 0$ its boundary. During the degradation phase of interest, a fault drives the margin downward, and

$$t_0 = \inf\{t : M(t) \leq 0\}$$

denotes the expiry of the positive margin. All infima are taken over the monitored time domain with $\inf \varnothing = +\infty$; for the timing analysis I assume that t_0 is finite and attained. Let t_f be the physical deadline by which a recovery must have taken effect. I assume that the monitored margin is conservative, so that $t_f \geq t_0$, and define the post-expiry slack $\delta = t_f - t_0 \geq 0$.

A recovery triggered at t_{trig} requires duration $\tau_r > 0$ and succeeds if

$$t_{\text{trig}} + \tau_r \leq t_f. \tag{1}$$

Equivalently, $t_{\text{pnr}} = t_f - \tau_r$ is the latest safe trigger time, or point of no return, for that recovery.

I compare two trigger rules. A fixed-margin reactive rule fires at

$$t_{\text{trig}} = \inf\{t : M(t) \leq \rho\},$$

where the pad $\rho \geq 0$ has the units of M. The predictive rule estimates the degradation rate $\hat{r}(t) = -\widehat{\dot{M}}(t)$ and, when that estimate is positive, fires when the projected time to expiry is at most a horizon $T_h \geq 0$:

$$\begin{aligned} t_{\text{trig}} = \inf\big\{t : M(t) \leq 0 \vee \\ [\hat{r}(t) > 0 \ \wedge \ M(t) \leq T_h\, \hat{r}(t)]\big\}. \end{aligned} \tag{2}$$

The condition $M(t) \leq T_h \hat{r}(t)$ is the projected-time test $M(t)/\hat{r}(t) \leq T_h$ multiplied through by $\hat{r}(t) > 0$. An unavailable or non-positive rate estimate therefore leaves only the unpadded condition $M(t) \leq 0$. In both simulators of Section 5, the margin is a time headway measured in seconds, so the pad ρ is also in seconds and the degradation rate is dimensionless.

4 When extra lead can help

I first show that the predictive rule is a reactive rule with a rate-scaled pad, then determine when extra lead can change a recovery outcome, and finally compare the lead that each rule must spend to recover in every case.

Proposition 1 (Predictive triggering is a rate-scaled margin)**.** *On its positive-rate branch, the predictive rule* (2) *is a reactive threshold with the time-varying pad $\rho(t) = T_h\hat{r}(t)$; its fallback pad is zero. Suppose that, on a monitored interval containing both threshold crossings, $M(t) = r(t_0 - t)$ with an exact constant estimate $\hat{r} = r > 0$, and that monitoring begins before either crossing. Then the predictive rule fires T_h seconds before t_0, whereas a fixed pad ρ fires ρ/r seconds before t_0. At one known constant rate, the fixed pad $\rho = T_h r$ therefore reproduces the predictive trigger exactly; for a positive horizon and two distinct rates, no single fixed pad provides the same lead at both rates.*

Proof. The threshold identity on the positive-rate branch follows directly from (2). Under the affine model, $M(t_0 - L) = rL$, so the predictive threshold $M = T_h r$ is reached at lead $L = T_h$ and the fixed threshold $M = \rho$ at $L = \rho/r$; the monitoring assumption makes these the first crossings. If $T_h > 0$ and $r_1 \neq r_2$, matching the lead at both rates would require both $\rho = T_h r_1$ and $\rho = T_h r_2$, which is impossible. □

Proposition 2 (When lead changes the recovery outcome)**.** *Suppose that a predictive rule triggers $L \geq 0$ seconds before t_0 and the unpadded reactive rule triggers at t_0. Then:*

1. *if $\tau_r \leq \delta$, both rules recover in time;*
2. *if $\delta < \tau_r \leq \delta + L$, only the predictive rule recovers in time; and*
3. *if $\tau_r > \delta + L$, neither rule recovers in time.*

Proof. By (1), the unpadded rule succeeds exactly when $t_0 + \tau_r \leq t_f$, that is, $\tau_r \leq \delta$. The predictive rule succeeds exactly when $t_0 - L + \tau_r \leq t_f$, that is, $\tau_r \leq \delta + L$. The three cases follow. □

Fig. 1 illustrates the timing argument. The same argument shows how observation latency enters.

Corollary 1 (Observation age)**.** *Suppose that a trigger rule acts on margin observations of age $\Delta \geq 0$, so that a rule with lead L on fresh observations fires at $t_0 - L + \Delta$. It recovers in time if and only if $\tau_r + \Delta \leq \delta + L$. Under the affine, exact-rate model of Proposition 1, the age delays the fixed-pad and the predictive triggers by the same amount.*

Proof. Substituting $t_{\text{trig}} = t_0 - L + \Delta$ into (1) gives the condition. Under the affine model, the margin observed at time t is $M(t - \Delta) = r(t_0 - t + \Delta)$, so each threshold in Proposition 1 is crossed Δ seconds later than with fresh observations. □

Observation age therefore acts as additional recovery time. Latency removed from the monitoring path, for example by computing the margin on board rather than remotely, enlarges the set of recoverable cases exactly as an equal gain in predictive lead does. Because age shifts both rules equally, it changes whether any trigger is early enough but not which rule is earlier.

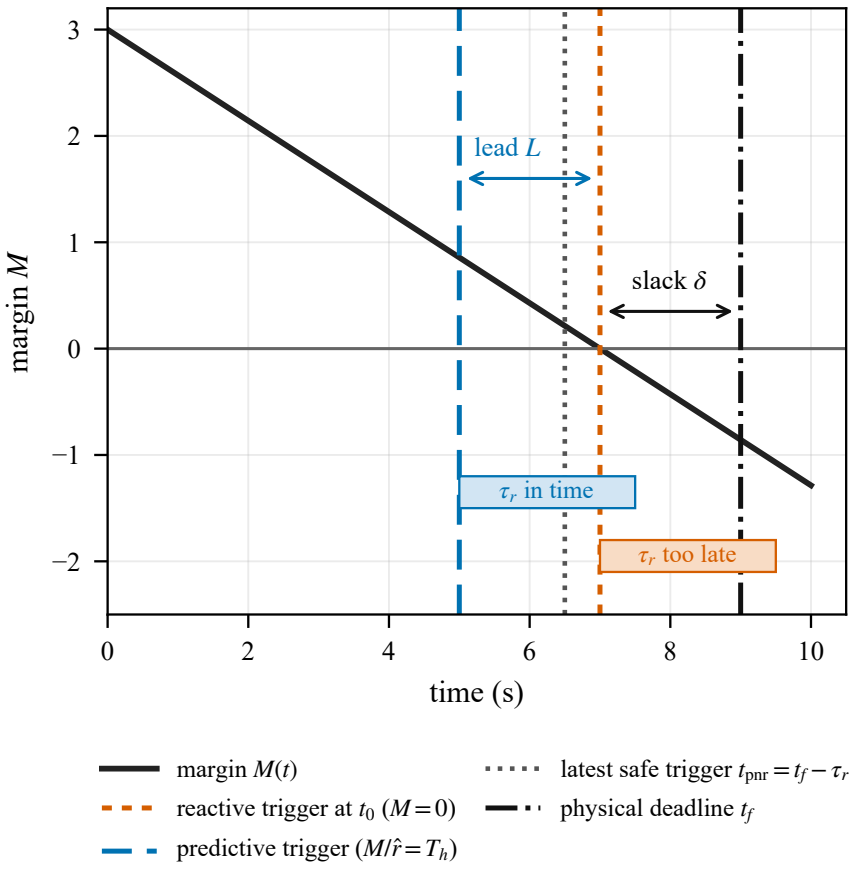


Figure 1: Timing model. The unpadded reactive rule fires at margin expiry t_0, whereas the predictive rule fires L seconds earlier. The earlier trigger changes the recovery outcome only when $\delta < \tau_r \leq \delta + L$.

Lead is not free. Acting earlier than recovery requires lengthens waiting and triggers interventions in situations that would have resolved without them. To compare the two rules across heterogeneous situations, consider cases $k = 1, \dots, K$ with rates $r_k > 0$ under the affine, exact-rate model, slack δ_k, and recovery duration $\tau_{r,k}$. By Proposition 2, a rule with lead L_k recovers in case k if and only if $L_k \geq L_k^\star$, where $L_k^\star = \tau_{r,k} - \delta_k$ is the *required lead*; the corresponding *required margin* is $m_k^\star = r_k L_k^\star$. I call $L_k - L_k^\star$ the *excess lead* of a rule that recovers in case k.

Proposition 3 (Homogeneity of the recovery requirement)**.** *Assume $L_k^\star > 0$ for all k. The smallest fixed pad that recovers in every case is $\rho^\star = \max_k m_k^\star$, and the smallest horizon that does so is $T_h^\star = \max_k L_k^\star$. With these choices, the excess lead in case k is*

$$e_k^{\text{fix}} = \frac{\rho^\star - m_k^\star}{r_k} \quad \textit{and} \quad e_k^{\text{pred}} = T_h^\star - L_k^\star. \tag{3}$$

Hence the fixed pad wastes no lead in any case if and only if the required margins are equal across cases, and the predictive horizon wastes no lead in any case if and only if the required leads are equal. If all rates are equal, the two rules have identical excess lead in every case.

Proof. By Proposition 1, the fixed pad gives lead ρ/r_k and the horizon gives lead T_h. The pad recovers in every case if and only if $\rho \geq r_k L_k^\star = m_k^\star$ for all k, and the horizon does so if and only if $T_h \geq L_k^\star$ for all k, which gives the minimal values. Substitution gives (3), which vanishes for every k exactly when the corresponding requirement is constant. If $r_k = r$ for all k, then $\rho^\star = r \max_k L_k^\star$ and $e_k^{\text{fix}} = T_h^\star - L_k^\star = e_k^{\text{pred}}$. □

Proposition 3 turns the question in the title into a question about the recovery requirement. Prediction is structurally preferable when recovery needs a similar amount of *time* in every case but, because degradation rates differ, a different amount of *margin*. A fixed pad is preferable when the required margin is homogeneous instead. Excess lead is only a proxy for the cost of acting early; the simulations measure that cost directly through elapsed time and unnecessary interventions.

Corollary 2 (Necessary conditions for an advantage over fixed pads)**.** *Within the deadline model, an earlier trigger converts a recovery failure into success only in the middle band of Proposition 2, shifted by any observation age as in Corollary 1; this requires positive lead. Within the affine, exact-rate model, the minimal horizon can waste less lead than the minimal safe fixed pad only if degradation rates vary across cases (Proposition 3). These conditions are necessary within the stated models but not sufficient: rate-estimation error and the joint distribution of rates, slack, recovery durations, and observation age can still make a fixed pad preferable.*

5 How I tested the idea

5.1 Single-Margin Simulator

Two robots travel on perpendicular paths through one crossing, and robot A may brake and wait for robot B. The monitored margin is a time headway, $M = |\hat{g}| - k_\sigma \sigma - G_{\text{safe}}$, where $\hat{g}$ is the estimated difference between the arrival times of B and A at the crossing, σ is a range-dependent uncertainty allowance, and $G_{\text{safe}} = 0.5\,\text{s}$ is the required headway; additive Gaussian noise perturbs each measurement. An optimistic hidden bias sf^p delays B's estimated arrival, where s is the fault severity in seconds, $f \in [0, 1]$ is B's remaining range fraction, and the exponent p sets how the bias decays: $p < 1$ reveals the conflict late, $p = 1$ linearly, and $p > 1$

early. Recovery is rate-limited braking with capability $a_{\max}$. The predictive rule estimates the margin slope with the Theil–Sen estimator [19] over the eight most recent samples. To avoid dividing by a fitted slope near zero, it projects only when the fitted decay exceeds 10^{-3} margin units per second (10^{-6} in the coordination simulator) and otherwise uses its $M \leq 0$ fallback.

Two model-specific quantities bound the latest useful trigger; neither is a general safety certificate. A continuous-time braking calculation gives the latest trigger that stops A outside the wider crossing zone, 8.65 s with the default parameters; it is conservative relative to the simulator's collision condition. Deterministic replay on the simulator's 0.1 s control grid finds 8.8 s as the latest tested forced-trigger time that avoids a collision in the default conflict episode; this boundary is empirical and grid-dependent.

5.2 Three-Robot Coordination Simulator

Three robots approach one shared zone at a nominal speed of 1.2 units/s; the zone is one unit long, so the nominal occupancy is 0.83 s. In the conflict condition, start distances of $(12, 12.5, 13)$ units put adjacent nominal entry times only 0.42 s apart, so untreated trajectories overlap physically. In the clear condition, start distances of $(12, 14, 16)$ units separate entries by 1.67 s. A collision is recorded when interpolated zone-occupancy intervals overlap, and a separate headway violation when two entry times are less than 5/6 s apart. Each robot's estimated arrival time includes a decaying hidden bias with per-robot multipliers $(0, 1, 2)$ and the exponent family used above. For each pair of robots, the monitored margin is the absolute difference between their estimated arrival times, minus the required headway and both uncertainty allowances, plus measurement noise. The fixed rule triggers when any pairwise margin is at most ρ. The predictive rule keeps a separate Theil–Sen slope history for each pair and triggers when a margin is nonpositive or its projected time to zero is at most T_h.

Both rules invoke the same recovery, an explicit one-shot serialization state machine. Robots start in the APPROACHING state. At the first trigger, any robot already CROSSING is placed first, the others are ordered by their current estimated arrival times, and the order is committed for the episode. Every later robot enters WAITING with all earlier robots as blockers and returns to APPROACHING only after every blocker is CLEARED. Pairwise slope histories are reset at commitment, and no robot yields twice. The design isolates the effect of trigger timing on a fixed recovery; it is not a general multi-robot planner. Table 1 lists the main parameters of both simulators.

Table 1: Simulator and Evaluation Parameters

Parameter	Single margin	Coordination
Robots	2	3
Time step	0.1 s	0.1 s
Nominal speed	1.2 units/s	1.2 units/s
Required headway	0.5 s	5/6 s
Measurement-noise s.d.	0.05 s	0.05 s
Slope window	8 samples	8 samples
$a_{\max}$ (units/s^2)	$0.5, 1.0, 2.5$	$0.5, 1.0, 2.5$
Degradation exponent p	$0.7, 1.0, 1.5$	$0.7, 1.0, 1.5$
Fault severity s (s)	$2, 4, 6, 8, 10$	$2, 6, 10$
Conflict scenarios	45	27
Pad and horizon grid	0–4, step 0.25	0–4, step 0.25
Tuning / test replicates	25 / 150	20 / 60
Held-out conflict episodes	6,750 per arm	1,620 per arm

5.3 Tuning and testing

Exploratory sweeps. The dose-response experiment compares an uncertainty-blind arrival-gap trigger, the current-margin trigger at $M = 0$, and the predictive rule with $T_h = 2.0\,\mathrm{s}$, using 40 paired noise seeds at each of seven severities. The recovery-capability and degradation-shape sweeps use 25 paired seeds at each of five severities, or 125 episodes per plotted setting and rule; the shape sweep fixes $a_{\max} = 0.8$. For each setting, the fixed-pad candidates are $\rho \in \{0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0\}\,\mathrm{s}$, and the displayed comparator is the candidate with the lowest mean successful elapsed time among those for which every episode succeeds on the same setting and seed grid. Selection and reporting therefore use the same simulations, so these figures are mechanism checks rather than out-of-sample estimates.

Seed-disjoint single-margin evaluation. The evaluation crosses three recovery capabilities, three degradation exponents, and five severities, giving the 45 conflict scenarios of Table 1. The fixed pad and the predictive horizon are both searched from 0 to 4 in steps of 0.25. Each candidate receives 25 tuning replicates per scenario (1,125 episodes). Among candidates with no tuning collision, selection minimizes the mean conflict elapsed time and breaks ties toward the smaller parameter. Testing then uses 150 new replicates per scenario, or 6,750 episodes per conflict arm. Clear-path behavior is evaluated at $a_{\max} = 1.0$ over the 15 exponent–severity combinations, for 2,250 episodes per arm; a yield in this condition is a false yield. The original $T_h = 2.0\,\mathrm{s}$ rule is retained as a diagnostic arm, but the primary comparison uses the separately tuned horizon.

Seed-disjoint coordination evaluation. Tuning crosses the same three capabilities and exponents with three severities, giving 27 scenarios, and searches both parameters over the same grid. Each candidate receives 20 tuning replicates per scenario (540 episodes). A candidate qualifies only if every tuning episode reaches the exit without collision, headway violation, timeout, or serialization breach; selection minimizes the mean successful makespan and breaks ties toward the smaller parameter. Each selected rule and a no-intervention reference then receive 60 new replicates per scenario and condition, or 1,620 episodes.

Seeds. Every split–scenario–replicate tuple maps to a distinct deterministic seed, giving 10,125 single-margin and 3,780 coordination seeds with no overlap between tuning and testing. A tuple's seed is shared across candidates and rules to preserve pairing. The split therefore holds out noise realizations only; the simulators and scenario grids remain fixed.

5.4 How I report uncertainty

Failure counts are reported before any timing comparison. For each exact scenario, I compute a two-sided 95% Wilson score interval for the collision probability [20]. I report the largest upper endpoint across the pointwise intervals; this maximum is neither a simultaneous confidence bound nor an estimate for a population of scenarios, and pooled counts summarize a fixed factorial design rather than one homogeneous binomial experiment. Elapsed time and makespan are summarized only over successful episodes. A paired percentile bootstrap with 20,000 resamples, clustered by test replicate [21], is computed only when every compared pair succeeds. If failure outcomes differ, I report descriptive successful-only means and suppress the inferential timing comparison.

5.5 Connection to the Timing Model

The recovery-capability sweep varies $a_{\max}$ as a proxy for recovery duration, and the exponent sweep varies how quickly the margin degrades. The simulations do not measure the episode-level distributions of τ_r, δ, L, and Δ, and they introduce no observation delay. They test behavior consistent with the timing argument but do not validate Propositions 2 and 3 by measuring all of their variables.

6 What the simulations show

6.1 Exploratory Sweeps

Untuned baselines favor early triggering (Fig. 2). Against an uncertainty-blind arrival-gap rule and an unpadded current-margin rule, the predictive rule with $T_h = 2.0\,\text{s}$ appears strong. With rate-limited recovery ($a_{\max} = 1.0$), the uncertainty-blind rule fails first as severity grows, the unpadded rule collides in many moderate- and high-severity episodes, and the predictive rule records no collision on this exploratory seed grid. Earlier action can therefore matter in the simulator, but this does not show that prediction is needed, because a padded current-margin rule can also act early.

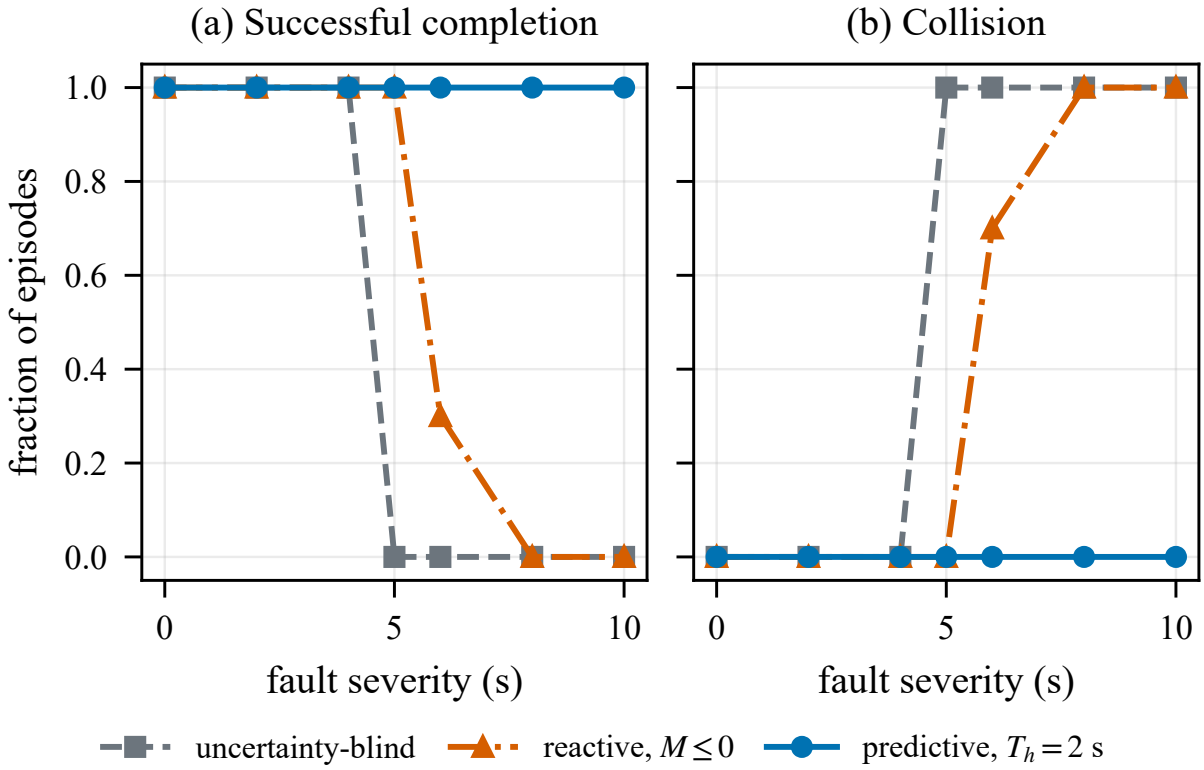


Figure 2: Exploratory single-margin dose response with $T_h = 2.0\,\text{s}$ and $a_{\max} = 1.0$: (a) successful completion rate and (b) collision rate over 40 paired noise seeds per severity. Fig. 3 adds an in-sample fixed-pad comparator.

The observed boundary is narrow (Fig. 3). The plotted difference is the mean elapsed time of the selected fixed pad minus that of the predictive rule, so positive values favor prediction, and the fixed pad is selected in-sample at every setting. Under the slowest recovery ($a_{\max} = 0.5$),

the predictive rule finishes about 0.56 s sooner; the sign changes as recovery becomes faster, reaching −1.31 s at $a_{\max} = 3.0$. In the degradation-shape sweep, the predictive rule records collisions at $p = 0.3$ and $p = 0.5$, where no tested fixed pad succeeds in every episode either; it is about 1.23 s faster at $p = 0.7$ and slower than the selected fixed pad from $p = 0.9$ onward. These differences are small relative to episodes of roughly 25–30 s, and the in-sample selection supports no inferential claim.

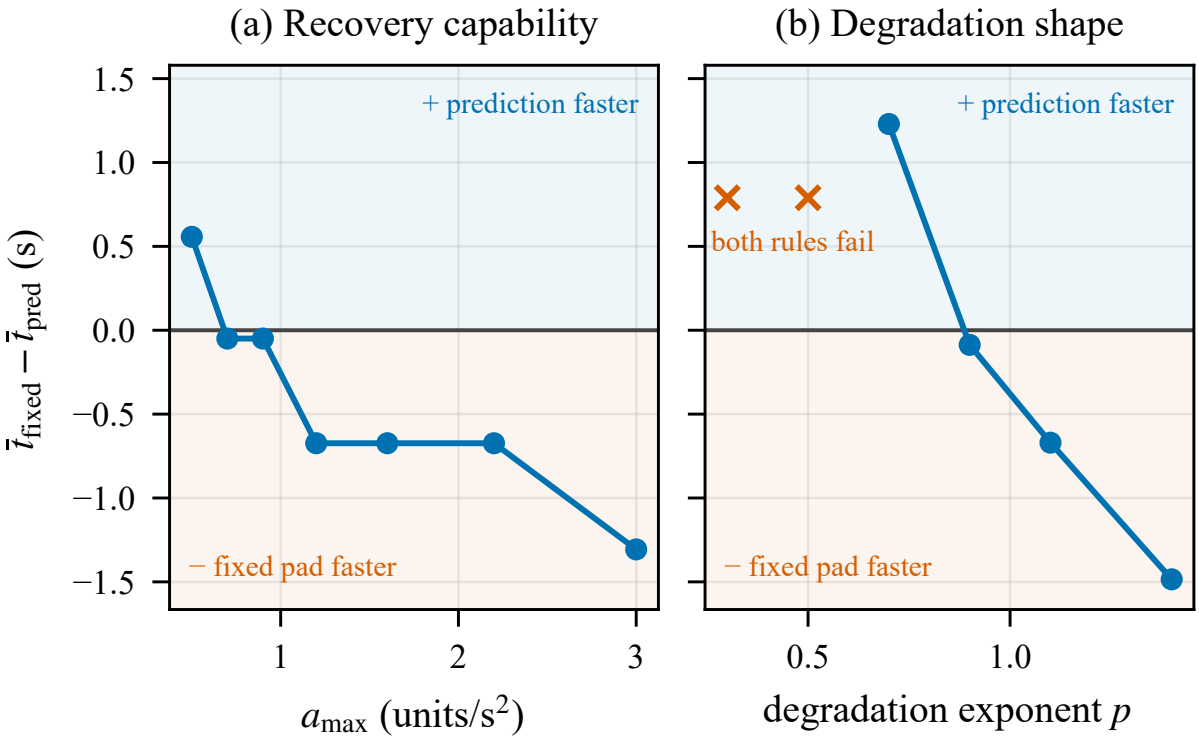


Figure 3: Exploratory elapsed-time difference between the selected fixed pad and the $T_h = 2.0$ s predictive rule across (a) recovery capability and (b) degradation shape at $a_{\max} = 0.8$. Positive values favor prediction. Differences are shown only where both rules recorded no failure; crosses mark settings in which the predictive rule collides and no tested fixed pad succeeds in every episode.

6.2 Seed-Disjoint Single-Margin Evaluation

The equal-budget searches select $\rho = 2.5$ s and $T_h = 2.5$ s (Table 2). On the 6,750 held-out conflict episodes, the fixed rule succeeds in all episodes and records no collision. The tuned predictive rule succeeds in 6,748 and records two collisions, one at each of the two highest severities with $a_{\max} = 0.5$ and $p = 0.7$. Its successful-only mean elapsed time is 26.577 s, compared with 27.690 s for the fixed rule, but these means condition on different success sets. I therefore attach no paired interval and claim no efficiency advantage. For context, the retained $T_h = 2.0$ s rule records 172 collisions on the same held-out set.

In the 2,250 held-out clear episodes, both tuned rules complete without collision. The fixed rule yields unnecessarily in 1800/2250 episodes (80.0%), and the predictive rule in 1358/2250 (60.4%). This is evidence of fewer interventions on the specified clear grid, not a safety-adjusted performance gain. The largest upper endpoint of the pointwise 95% Wilson collision intervals is 0.025 for the fixed rule and 0.037 for the predictive rule.

Table 2: Seed-Disjoint Single-Margin Test Results over 6,750 Conflict Episodes per Arm

Rule	Parameter	Collisions	Successes	Mean time (s)
Fixed pad, tuned	$\rho = 2.5\,\mathrm{s}$	0	6,750	27.690
Predictive, tuned	$T_h = 2.5\,\mathrm{s}$	2	6,748	26.577
Predictive, retained	$T_h = 2.0\,\mathrm{s}$	172	6,578	25.990

Mean elapsed time is averaged over successful episodes only and therefore conditions on different success sets for different rules.

6.3 Seed-Disjoint Coordination Evaluation

The redesigned geometry first passes a falsification check: without intervention, all 1,620 held-out conflict runs reach the exit but contain both a zone-occupancy collision and a headway violation, so none succeeds (Table 3). These runs consume no measurement noise and are therefore deterministic repeats within each physical scenario; their count verifies coverage of the grid, not 1,620 independent safety trials.

Table 3: Seed-Disjoint Coordination Test Results over 1,620 Episodes per Rule and Condition

	No intervention	Fixed pad $\rho = 1.75\,\mathrm{s}$	Predictive $T_h = 1.50\,\mathrm{s}$
Conflict condition			
Successful episodes	0	1,620	1,620
Collisions	1,620	0	0
Headway violations	1,620	0	0
Mean makespan (s)	–	29.813	27.034
Mean first trigger (s)	–	4.534	5.917
Clear condition			
Successful episodes	1,620	1,620	1,620
Serializations	0	1,080	21
Mean makespan (s)	20.000	26.337	20.278

Makespan is averaged over successful episodes. No rule records a timeout, repeated yield, or serialization breach. The paired fixed-minus-predictive conflict difference is 2.779 s, with a 95% interval of $[2.659, 2.902]$ s from a bootstrap clustered by test replicate.

The searches select $\rho = 1.75\,\mathrm{s}$ and $T_h = 1.50\,\mathrm{s}$, each with 540 of 540 successful tuning episodes and no serialization breach. Each selected rule then succeeds in all 1,620 held-out conflict episodes with no collision, headway violation, timeout, repeated yield, or serialization breach. The mean successful makespan is 29.813 s for the fixed rule and 27.034 s for the predictive rule, whose mean first trigger occurs later (5.917 versus 4.534 s) and so avoids some waiting without changing the observed success count. The paired fixed-minus-predictive difference is 2.779 s, with a 95% interval of $[2.659, 2.902]$ s from a bootstrap that clusters the 27-scenario grid by the 60 test replicates. This is an efficiency comparison at matched observed outcomes, conditional on the modeled scenarios and noise distribution.

In the clear condition, every run without intervention succeeds in 20.000 s, and both selected rules also succeed in all 1,620 episodes. The fixed rule commits an unnecessary serialization in 1080/1620 episodes (66.7%) and has a mean makespan of 26.337 s; the predictive rule serializes in 21/1620 (1.3%) and has a mean of 20.278 s. No interval was computed for this clear-path difference, so it remains descriptive.

7 What the findings mean

7.1 Questions to ask before adding prediction

Before adding prediction, I would ask whether its extra lead changes what recovery can accomplish. Within the timing model, four questions help. First, does recovery need lead at all, that is, does some case have $\tau_r > \delta$? If not, an unpadded reactive rule suffices (Proposition 2). Second, how old are the observations? Age consumes lead one for one, so reducing latency in the monitoring path can be as valuable as prediction (Corollary 1). Third, is the recovery requirement homogeneous in time or in margin? If recovery needs a similar lead in every case while degradation rates differ, the predictive horizon wastes less lead than any safe fixed pad; if the required margin is similar instead, a fixed pad suffices (Proposition 3). Fourth, can the rate be estimated quickly and accurately enough from the available observations? Proposition 1 also suggests an intermediate design: scale a current-margin threshold with an online rate estimate instead of constructing a full predict-then-recover stack. Its value would still depend on estimator delay, noise, and changes in the degradation rate.

7.2 Why the two studies differ

The two simulators produce different results. In the single-margin study, the exploratory figures favor prediction over the unpadded rules and, in a few settings, over the in-sample fixed pad. After equal-budget, seed-disjoint tuning, the predictive rule intervenes less often on clear paths and has a lower descriptive successful-only elapsed time, but it also records two held-out conflict collisions where the fixed rule records none, so no efficiency comparison at matched outcomes is available. Both collisions occurred in the slowest-recovery, late-reveal corner of the grid ($a_{\max} = 0.5$, $p = 0.7$), where the margin decline steepens as the conflict approaches. A slope fitted over the eight most recent samples then lags the current rate and shortens the realized lead, which is consistent with the exact-rate assumption of Proposition 1 being violated where it matters most.

In the coordination study, both tuned rules succeed throughout the held-out conflict grid, and the predictive rule finishes earlier and serializes far fewer clear paths. This pattern is consistent with Proposition 3 when degradation rates vary across robot pairs and scenarios: a fixed pad that is safe for the fastest degradation must trigger early, and often needlessly, when degradation is slow or absent. The result depends on the recovery design. The first trigger commits one total order, after which blockers and release events determine the motion; prediction changes when this commitment occurs, but the order is never revised. The paired interval describes variation over measurement noise on the fixed 27-scenario grid.

7.3 Scope of the experiments

I used two-dimensional point-kinematic models to isolate trigger timing, with Gaussian measurement noise and braking or serialization as recovery. The held-out tests use new noise realizations on the same scenario grids. The coordination model has three robots, one shared zone, and a crossing order that stays fixed after the first trigger. Testing vehicle dynamics, perception, network delays, and online replanning would extend the study toward a deployed robot system.

The exploratory plots select the fixed pad in-sample and use a 2.0 s predictive horizon; the held-out comparisons tune both rules separately. The 8.8 s replay deadline belongs to one

deterministic episode on a 0.1 s trigger grid, while the analytical stopping deadline uses a conservative braking calculation. The related-work search was targeted rather than systematic.

7.4 What I would test next

I would start with three tests. First, injecting observation delay into both simulators would test Corollary 1 and quantify how latency trades against predictive lead. Second, varying recovery durations jointly with degradation rates would test Proposition 3 by measuring the required lead and margin per case. Third, experiments on ROS 2 with the Nav2 navigation stack, with localization degradation, communication faults, planner interactions, and full-stack telemetry, would test whether the boundary survives realistic estimation and middleware timing.

8 What I take from this

For the trigger studied here, prediction is a way to adapt a margin threshold to the estimated rate of decline. Its value depends on whether the extra lead changes what recovery can accomplish and how much unnecessary early action it avoids. In the timing model, recovery duration and deadline slack set that boundary, and observation age consumes the available lead one for one.

The three-robot experiment shows a practical benefit of this adaptation. Both tuned rules succeed in all 1,620 conflict episodes, while prediction reduces mean completion time by 2.779 s (paired 95% interval $[2.659, 2.902]$ s). It also cuts unnecessary clear-path serializations from 1,080 to 21 out of 1,620 episodes. Waiting until closer to the point when recovery is needed saves time in this fixed-order crossing setup.

The two-robot experiment shows why the rate estimate matters. The tuned fixed rule records 0/6750 conflict collisions and prediction records 2/6750, both where the conflict is revealed late and braking is slow. A rate estimate that lags a steepening decline can use up the lead the predictive rule was meant to provide.

My takeaway is that the recovery requirement is the right place to start. When recovery needs similar lead times across cases but margins decline at different rates, adapting the threshold can reduce unnecessary waiting. Observation delay and changing recovery durations are the next variables I would test, using the tuned fixed threshold as the baseline.

Code and data. The simulators, tests, episode-level results, and figure scripts are available at `https://github.com/fishOmlette/X`.

References

[1] A. Desai, S. Ghosh, S. A. Seshia, N. Shankar, and A. Tiwari, "SOTER: A runtime assurance framework for programming safe robotics systems," in *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2019, pp. 138–150.

[2] C. Tsai and S. Hariri, "Mission-level runtime assurance framework for autonomous driving," arXiv:2606.06996, 2026.

[3] N. Kekatos, P. Katsaros, A. Lekidis, T. Nestoridis, and T. Nianios, "Mission-level runtime assurance for LLM-assisted ISR swarms over a verification-aware fabric," arXiv:2607.23532, 2026.

[4] S. Pinisetty, T. Jéron, S. Tripakis, Y. Falcone, H. Marchand, and V. Preoteasa, "Predictive runtime verification of timed properties," *Journal of Systems and Software*, vol. 132, pp. 353–365, 2017.

[5] S. Pinisetty, V. Preoteasa, S. Tripakis, T. Jéron, Y. Falcone, and H. Marchand, "Predictive runtime enforcement," *Formal Methods in System Design*, vol. 51, no. 1, pp. 154–199, 2017.

[6] H. Yoon, Y. Chou, X. Chen, E. Frew, and S. Sankaranarayanan, "Predictive runtime monitoring for linear stochastic systems and applications to geofence enforcement for UAVs," in *Runtime Verification (RV 2019)*, ser. Lecture Notes in Computer Science, vol. 11757. Springer, 2019, pp. 349–367.

[7] B. Wheatman, J. Chen, T. Sookoor, and Y. Amir, "RADICS: Runtime assurance of distributed intelligent control systems," in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, 2021, pp. 182–187.

[8] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," *Automatica*, vol. 129, 2021, Art. no. 109597.

[9] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.

[10] U. Mehmood, S. Sheikhi, S. Bak, S. A. Smolka, and S. D. Stoller, "The black-box simplex architecture for runtime assurance of autonomous CPS," in *NASA Formal Methods (NFM 2022)*, ser. Lecture Notes in Computer Science, vol. 13260. Springer, 2022, pp. 231–250.

[11] D. Bozhinoski, D. Garlan, I. Malavolta, and P. Pelliccione, "Managing safety and mission completion via collective run-time adaptation," *Journal of Systems Architecture*, vol. 95, pp. 19–35, 2019.

[12] D. N. Lee, "A theory of visual control of braking based on information about time-to-collision," *Perception*, vol. 5, no. 4, pp. 437–459, 1976.

[13] S. M. S. Mahmud, L. Ferreira, M. S. Hoque, and A. Tavassoli, "Application of proximal surrogate indicators for safety evaluation: A review of recent developments and research needs," *IATSS Research*, vol. 41, no. 4, pp. 153–163, 2017.

[14] N. Xu, Z. Yang, H. Zhuo, L. Liao, Y. Sui, and N. He, "Risk-aware adaptive safety margins for model predictive control with orientation–motion coupled barrier functions in dynamic environments," *Actuators*, vol. 15, no. 2, 2026, Art. no. 116.

[15] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *2019 18th European Control Conference (ECC)*, 2019, pp. 3420–3431.

[16] A. K. S. Jardine, D. Lin, and D. Banjevic, "A review on machinery diagnostics and prognostics implementing condition-based maintenance," *Mechanical Systems and Signal Processing*, vol. 20, no. 7, pp. 1483–1510, 2006.

[17] V. Nigam and C. Talcott, "Automating recoverability proofs for cyber-physical systems with runtime assurance architectures," in *Theoretical Aspects of Software Engineering (TASE 2023)*, ser. Lecture Notes in Computer Science, vol. 13931. Springer, 2023, pp. 1–19.

[18] G. Marín and J. Chaudhary, "Governing what you cannot observe: Adaptive runtime governance for autonomous AI agents," arXiv:2604.24686, 2026.

[19] P. K. Sen, "Estimates of the regression coefficient based on Kendall's tau," *Journal of the American Statistical Association*, vol. 63, no. 324, pp. 1379–1389, 1968.

[20] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[21] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, ser. Monographs on Statistics and Applied Probability. New York, NY, USA: Chapman & Hall, 1993, vol. 57.